Why Are GUI Agents Correct but Late? Decode on the Decision-Time Critical Path, Tested with Pre-Compiled Policy Trees

Zihan Dong, Rui Qian, Qishi Zhan, Dongshen Peng, Kaixin Li, Yu Li

Paper version: arXiv 2607.28399v1 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *The typical failure is not wrong understanding but late understanding.*

Full-model decoding sits on the decision-time critical path — a 567 ms p50 round trip vs ~600 ms windows.

Three falsifiable predictions

1. **Act early?** R1 open-loop, P1 replan — both score 0.
2. **Decode latency?** Gain once prep > planner latency.
3. **Raw speed?** No valid tree → no gain.

Evaluation setup

key_prompt · timed GUI | 3 keys · 42-pair blocks | 250–2000 ms windows | Holo-3.1-35B-A3B | DynaCU-Bench · 39 | no fine-tuning

- **Online:** server-side scoring; key before deadline.
- **Paired:** 42 pairs, exact McNemar, pre-registered seeds.

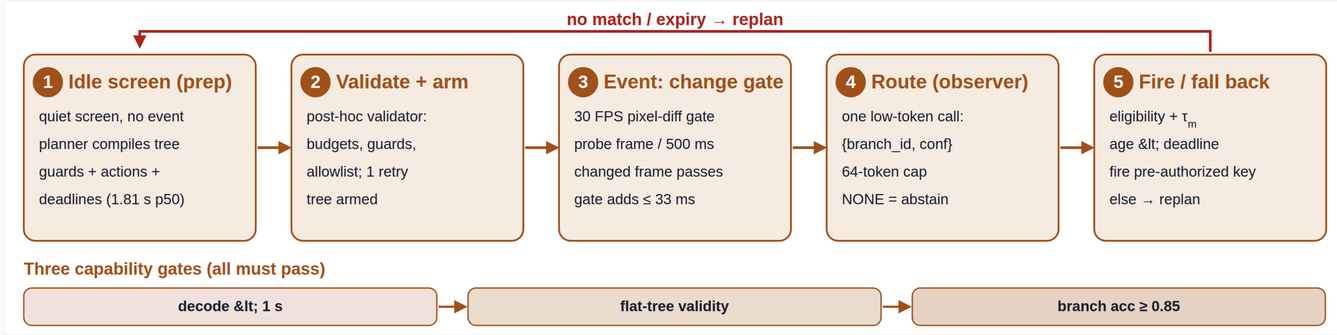
Our read (CUA Radar)

65	★★	online
score of 100	band	evidence level

- **Reports:** online eval; DynaCU transfer; replication; costs; ablation.
- **Flags:** AP15, gain in one model class; no code.
- Pattern C17 (manager–worker); authors' numbers, not rerun.

AAPT: compile in idle time, route at event time

- **Sees / acts:** 30 FPS screenshots via a pixel-diff change gate; allowlisted keypress primitives; desktop timed prompts.
- **Planner (idle screen):** the frozen model compiles a flat tree of guards, pre-authorized actions, branch deadlines (1.81 s p50).
- **Observer (event):** one 64-token JSON call picks a branch; eligibility, confidence floor, deadline checks; no match → replan.

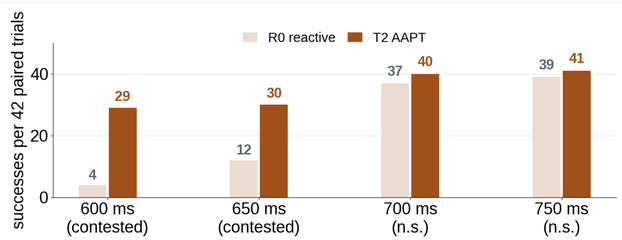


Our redraw (not the paper's figure): idle-time compile vs event-time route.

$$T_{\text{cover}} \geq L_{p95} + M$$

the tree must outlive the planner's own latency; margin M = 500 ms

Results: deadline success



Contested-window scan (Table 11): 42 pairs per window, exact McNemar.

650 ms primary: 0.79 vs 0.50, p = 1.8×10⁻³, zero wrong actions

SO WHAT → *500 ms floor: T2 28/42 vs R0 0/42.*

Ablation: branch budget (k × n)

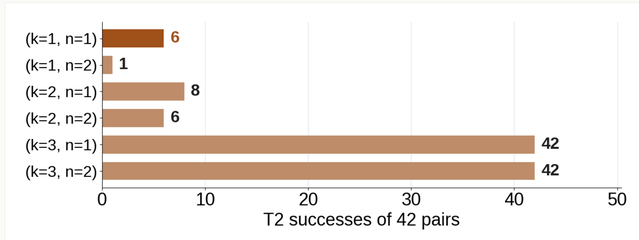


Table 7, redrawn: T2 successes of 42 pairs at 600 ms; shared reactive arm R0 = 0.548.

SO WHAT → *Knee at (k=3, n=1) = outcome-set size: 42/42, 19/0 discordants (p = 3.8×10⁻⁶); k < 3 fails on validity. Observer cap sweep: 32 truncates, 128 drifts, 64 wins (0.833). Mis-set budgets can fire wrong keys (13/42 at k=1).*

Headline Numbers

0.79
success, 650 ms window
vs 0.50 reactive; 42 pairs; incorrect actions 0

0.778 Qwen3.6 pooled

0% wrong actions

3.2x tokens vs reactive

The paper's own controls

CONTROL	RESULT	DECIDES
R1, P1 baselines	both 0 success	early ≠ enough
O1 oracle route	n.s. after Holm	observer binds
Prep sweep	0/42 at 1 s prep	gain from 2.4 s
6-model gates	all 3 must pass	speed alone: no
Cost (tokens)	T2 = 3.2× R0	planner off path

Takeaway

Pre-compiled guarded trees beat a reactive loop in contested windows (0.79 vs 0.50) — if outcomes can be listed in advance; where they cannot, reactive execution stays stronger.

planner 1.81 s, off path | observer ~325 ms p50 | wrong actions: 0 | external: tie 6/39

Scan to read the full paper

arxiv.org/abs/2607.28399v1